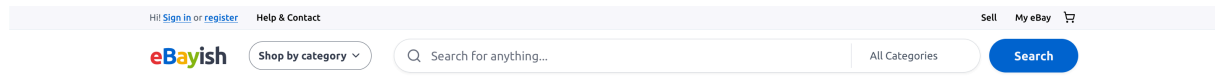


eBayish: An Online Bidding Platform



Buy.Sell. Discover.

Millions of items at incredible prices, just for you

Start Selling

Shop Now

Latest Listings



Shimano Dialuna 2.9m

📍 Balos

🕒 4d 4h 39m 18s left

🏆 0 bids

💰 Buy now for \$300.00



Greek Philosophers

📍 Manolates

🕒 5d 4h 39m 17s left

🏆 0 bids

💰 Buy now for \$500.00



MVD Predator Spearfishing gun 110cm

📍 Balos

🕒 3d 4h 39m 18s left

🏆 0 bids

💰 Buy now for \$350.00



Acoustic Guitar

📍 Manolates

💰 Buy now for \$280.00

Online Application Technologies Course

University Of Athens, 2024-2025

Contents

1. Introduction	3
2. Installation Guide	3
2.1. Docker Compose setup	3
2.2. Manual setup	3
3. Security	4
3.1. Authentication	4
3.2. Authorization	4
3.3. HTTPS/TLS	5
4. Database	6
5. Backend	6
5.1. General Architecture	6
5.2. Implementation Strategies	7
5.3. Users	8
5.4. Items	9
5.5. Auctions and Bids	9
5.6. Search	10
5.7. Real-Time Notifications	10
5.8. Real-Time Messaging	11
5.9. Recommendation System	11
6. Frontend	12
6.1. Frontend Architecture	12
6.2. State Management	13
6.3. Routing	13
6.4. API Integration	14
6.5. Real-Time Features	14
6.5.1. WebSocket Implementation	14
6.5.2. Real-Time Notifications	15
6.5.3. Real-Time Messaging	15
6.6. Key Feature Implementations	15
6.6.1. Authentication Flow	15
6.6.2. Item Listing and Management	15
6.6.3. Marketplace and Search	15
6.6.4. Bidding and Buying	15
6.6.5. Admin Dashboard	15

1. Introduction

During the course of this assignment, we created an online Auction/Bidding platform akin to Ebay. Ebayish provides a comprehensive set of features that is expected of an application like this. Some of the features that Ebayish implements are: a fully featured administration, with analytics and platform management tools, a listing creation page with images, location data, and support for auctions or “Buy It Now” listings, real-time bids, notifications, and messaging, marketplace search filters and a recommendation algorithm. All the requirements of the assignment have been met.

The application uses the client-server model. The server was built using the Actix Web framework for the Rust programming language. It offers a RESTful API that can be used by a variety of client applications. The website was created using React and TypeScript and functions as the client. In this documentation, we provide installation instructions along with a detailed explanation of the codebase and its architecture.

2. Installation Guide

There are two main ways to build and run the application. The recommended one is the docker compose setup because of it is agnostic to the build environment and easy to use. Both docker compose and manual setup will result in the application running with https on all of its connections (frontend, backend, database). Furthermore some data will be pre-seeded making basic functionality checking straight forward.

2.1. Docker Compose setup

For this method only docker and docker compose are needed. Navigating to the project's root directory and running `docker compose up`. Docker will download the necessary containers and build the application within them, note that the build may take up to 4 or 5 minutes depending on network speed and available resources. After the application is built it will be available on `https://localhost:5173`.

2.2. Manual setup

To setup this application manually we will need to install the following: `nodjs`, `npm`, `rustup`, `psql`, `sqlx-cli` and `docker`. First step to getting Ebayish up and running is the database. To setup the database we just need to run the script `./backend/scripts/init_db.sh`, this script will check for the existence of all dependencies required for initializing the database and running database migrations. After the database is up and running we can move on to building and running the backend as long as we are within the backend directory where the `Cargo.toml` file is located, and use the command `cargo run` this will build and run all the backend binaries, there is also the command, `cargo test` which builds and runs the automated testing suite. Finally for the frontend after navigating to the similarly named frontend directory, execute `npm i` this will install any frontend dependencies using the node package manager, after `npm` has finished using the command `npm run dev` will start the vite development server which will provide you with a localhost address within the terminal this is where the application is available, most likely `https://localhost:5173`.

3. Security

Security is fundamental to any online application, before going in to further detail, we want to highlight the fact that access controls have been implemented on both backend and frontend. The backend checks for the necessary authentication for use of privileged or role based endpoints, also doing obvious sanity checks and input sanitization while the frontend disable access to protected pages like the admin dashboard for non-admin users while also disabling several buttons to prevent undesired behavior.

3.1. Authentication

Meeting the assignment's specifications, this application uses JSON Web Tokens (JWTs) for authentication. Whenever a user signs up or logs in to the application, the backend first validates their username via input sanitization, then queries the database to retrieve relevant information like credentials and account status. It then proceeds to validate the password via Argon2, mints a new token that is then passed as a string back to the user's browser. This token can encapsulate several pieces of information in our case, we include the user's UUID, the username, their role, and a one-day expiration date is set. To improve security for a production environment, shorter token lifespans are advised, in the range of several minutes, but we omitted this for the purposes of this assignment to avoid the complexity of implementing a refresh token mechanism. Also, the JWT secret should be provided by environment variables and be declared as a secret within the source code, but in our app it's hardcoded to improve portability.

3.2. Authorization

For the authorization part of our application, we will show how we handle protected routes. The default behavior of endpoints is to allow all incoming requests, so we had to implement several checks and strategies for different endpoints.

The main technique used is JSON Web Token extraction. The Actix-Web framework provides a customizable extractor trait called `FromRequest`. This allows us to extract and validate JWT claims in all our handlers by simply including our custom `FromRequest` implementation called `Claims` in the given handler.

This is the definition of our `Claims` struct:

```
pub struct Claims {
    pub sub: String,
    pub username: String,
    pub role: String,
    pub exp: usize,
}
```

As you can see, this struct has fields to store all the deserialized data of our JWT. What follows is our custom implementation of The `FromRequest` trait provided by Actix-Web which populates the above struct.

```
impl FromRequest for Claims {
    type Error = Error;
    type Future = Ready<Result<Self, Self::Error>>;

    fn from_request(req: &HttpRequest, _: &mut Payload) -> Self::Future {
        let token = req
            .headers()
            .get("Authorization")
            .and_then(|h| h.to_str().ok())
            .and_then(|s| s.strip_prefix("Bearer "));
    }
}
```

```

match token {
  Some(token) => {
    let secret = b"opekepescam";
    let validation = Validation::default();
    match decode::<Claims>(
      token,
      &DecodingKey::from_secret(secret),
      &validation,
    ) {
      Ok(token_data) => ready(Ok(token_data.claims)),
      Err(_) => ready(
        Err(actix_web::error::ErrorUnauthorized("Invalid token"))
      ),
    }
  }
  None => ready(Err(actix_web::error::ErrorUnauthorized("Missing token"))),
}
}
}

```

By calling Claims as an input to a handler, it automatically calls the above code, either deeming the token valid and proceeding to other checks or throwing the appropriate error. Here we show a simple mock handler to demonstrate its use.

```

pub async fn is_user_admin(
  claims: Claims,
) -> Result<HttpResponse, actix_web::Error> {
  if claims.role != "admin" {
    return Err(actix_web::error::ErrorForbidden("Admin access required"));
  }
  Ok(HttpResponse::Ok().finish())
}

```

Other authorization checks, include extracting data from requests, in conjunction with the data provided to us by our Claims implementation, to make several sanity checks using either database queries, for further context when needed, or simple if-then-else statements.

Using the strategies highlighted above we tried to secure all edge cases we could think of, while also implementing other request validity checks. Rust and the Actix-Web framework made the process of implementing authorization highly customizable. It also provided a simple way to use the custom Claims trait in all necessary handlers, creating a clean yet effective pattern to achieve JWT extraction.

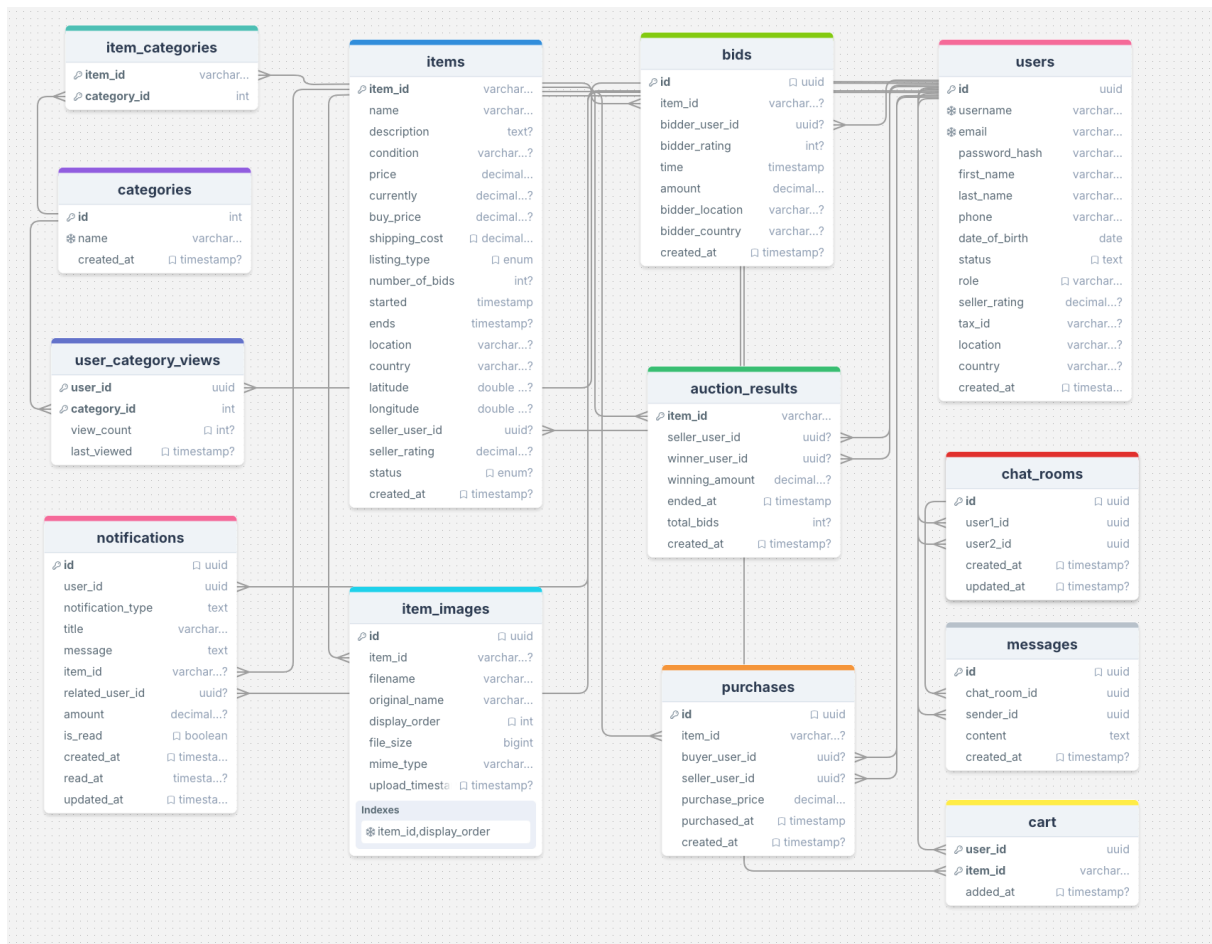
3.3. HTTPS/TLS

All connections in this application use HTTPS/TLS, which means that all database transactions, API calls, and frontend access are all encrypted. This greatly enhances the confidentiality and integrity of communication between the several parts of our application. Encrypting database transactions was the most straightforward, this was due to the fact that it is housed within a container, allowing us to enable SSL by simply including an environment variable. For API calls and frontend access we used rustls and vite respectively. Providing both with self-signed certificates and enabling their use within our configuration logic enabled us to secure those as well. Note that the self signed certificates will make the browser require further approval to navigate to the website.

4. Database

The database is an integral part of an application of this size. For this application we used a PostgreSQL relational database, we made that choice because of tools like `sqlx` and `diesel`, these tools allow for database interactions in Rust projects. In particular we used `sqlx` which allowed us to easily migrate and iterate through different schemas while, also providing compile time checks to allow for easy debugging. This functionality was necessary for efficient development of the backend routes allowing for a high degree of flexibility in the route logic.

Here you can see a schema of our database design.



5. Backend

We decided to use Rust for this project as a learning opportunity. Rust is not the most widespread backend language, but lately with the help of web frameworks like Actix-Web which was used here and Axium it has become more straight forward to create such applications. It was a great learning experience for us to work with a complex and modern language introducing new concepts like the borrow checker and the trait system. We also used `crates`(packages in Rust managed by Cargo the Rust package manager), for standardizing our error detection and logging strategies. In the following sections the architecture of the backend will be presented in greater detail.

5.1. General Architecture

The backend follows a classic three-layered architecture, which promotes separation of concerns and modularity. This design makes the codebase easier to understand, maintain, and test. The three layers are:

1. **Domain Layer:** This is the core of the application, containing the business objects and their logic. In our project, this layer is represented by the domain module, which defines the data structures like `Item`, `User`, `Bid`, etc.
2. **Service Layer:** This layer contains the business logic of the application. It orchestrates the application's functionality, working with the domain objects and coordinating with other services and the data access layer. The services module in our codebase represents this layer. For example, `auction_service.rs` contains the logic for managing auctions.
3. **Presentation Layer (Routes):** This is the outermost layer, responsible for handling incoming HTTP requests and sending responses. In our Actix-Web application, this is handled by the routes module. Each file in this module corresponds to a set of related API endpoints. For example, `items.rs` defines the routes for creating, retrieving, updating, and deleting items (CRUD). This layer delegates the actual business logic to the service layer.

This layered approach allows for a clean separation of responsibilities. The `main.rs` and `startup.rs` files are responsible for initializing the application, setting up the database connection pool, and configuring the Actix-Web server with its routes.

5.2. Implementation Strategies

The backend implementation leverages several modern Rust features and libraries to ensure robustness, performance, and maintainability.

- **Asynchronous Processing:** The entire web server is built on Rust's asynchronous capabilities using `async/.await`. Actix-Web is an asynchronous framework, meaning it can handle many concurrent connections efficiently without blocking threads. This is crucial for a responsive application, especially for features like real-time bidding and messaging.
- **Compile-Time Checked SQL:** We use `sqlx` for all database interactions. A key feature of `sqlx` is its ability to check SQL queries against the database schema at compile time. This prevents a whole class of runtime errors, such as typos in SQL queries or mismatches between the database schema and the data models in the code. This has greatly improved the development workflow and the reliability of the application.
- **Structured Logging:** For observability, we've implemented a structured logging system using the `tracing` and `tracing-bunyan-formatter` crates. This approach, configured in our `telemetry.rs` module, ensures that all log records are emitted as JSON. Structured logs are machine-readable, making it significantly easier to query, filter, and analyze log data, which is invaluable for debugging and monitoring in a production environment.
- **Declarative Error Handling with Macros:** This is one of the most beautiful implementations it is a centralized and declarative error handling mechanism, made possible by Rust's macro system. In our `error_handling.rs` module, we defined a macro, `define_route_error!`, that generates custom error types for our route handlers. This allows us to define all possible error states for a given domain, mapping each one to a specific HTTP status code and error message in a single, clean declaration.

Here is a snippet of the macro's definition:

```
#[macro_export]
macro_rules! define_route_error! {
    (
        $error_name:ident {
            $(
                $variant:ident => ($status:expr, $message:expr)
            )
        }
    )
}
```

```

        ),* $(,)?
    }
)
=> {
    // ... macro implementation ...
};
}

```

And here is how we use it to create a specific error type for user-related operations:

```

define_route_error!(
    UserError {
        ValidationError => (StatusCode::BAD_REQUEST, "Validation error"),
        InvalidCredentials => (StatusCode::UNAUTHORIZED, "Invalid credentials"),
        UserNotFound => (StatusCode::NOT_FOUND, "User not found")
    }
);

```

This approach drastically reduces boilerplate, eliminates the possibility of unhandled error states, and keeps our handler logic clean and focused on the business logic, letting the framework handle the conversion of errors into appropriate HTTP responses.

The codebase is organized into distinct modules (domain, services, routes), as described in the General Architecture section. This modularity is a key implementation strategy that promotes code reuse, simplifies testing, and makes the system easier to reason about.

5.3. Users

For the user management part of our application, we implemented a system that handles registration, and administrative actions, with a focus on security and data integrity.

A key part of this is the user lifecycle, which we manage with a status field in the database. A user's status can be pending, confirmed, or suspended. When a new user signs up, their account is pending until an administrator can review and verify it. Once verified, the status changes to confirmed, and the user gains full access. Administrators also have the tools to suspend a user's account and reactivate it later.

To ensure data is valid from the start, we used Rust's type system to our advantage. Instead of using simple String types for usernames and emails, we created our own Username and UserEmail types. This is known as the "newtype" pattern. This design choice is central to our validation strategy because it makes invalid states unrepresentable in our code. A Username object can only be constructed if the underlying string passes our validation rules. This is a powerful feature of Rust, as it provides guarantees at compile time. Any part of our code that receives a Username object can trust that it holds valid data, without needing to re-validate it.

Here is the validation logic within the Username type itself:

```

impl Username {
    pub fn parse(s: String) -> Result<Username, String> {
        let is_empty_or_whitespace = s.trim().is_empty();
        let is_too_long = s.graphemes(true).count() > 256;
        let forbidden_characters = ['/', '(', ')', '"', '<', '>', '\\', '{', '}'];
        let contains_forbidden_characters = s.chars().any(|g|
forbidden_characters.contains(&g));

        if is_empty_or_whitespace || is_too_long || contains_forbidden_characters {
            Err(format!("{s} is not a valid username."))
        } else {

```

```

        Ok(Self(s))
    }
}

```

This validation is then enforced at the entry point of our application, the route handler, preventing any invalid data from even reaching the service layer.

```

// From the create_user route handler
let username = Username::parse(value.0.username)?;

```

Finally, we designed the API endpoints for user management with clear role-based access. Public-facing endpoints only expose non-sensitive information, like a user's username and seller rating. The majority of functions, such as verifying or suspending users, are restricted to the admin.

5.4. Items

The core functionality of our platform revolves around items, which can be listed either for auction or as fixed-price sales. We have implemented a complete set of CRUD (Create, Read, Update, Delete) operations to manage these items.

When a user creates a new item, they must provide essential details such as the item's name, a description, price, and at least one category. A key distinction is made based on the `listing_type`.

- For a `fixed_price` item, the transaction is straightforward, similar to a standard e-commerce purchase.
- For a `auction` item, additional details are required, such as an end time for the auction. The system also tracks the current highest bid and the total number of bids.

To ensure data integrity, we have placed validation logic directly within our `NewItem` domain model. This `validate` method checks for several conditions before an item can be created or updated. For example, it ensures that an auction item has an end date and that its start time is before its end time. It also verifies that prices and shipping costs are not negative. This approach centralizes our business rules for items, making the system more robust and easier to maintain.

The API provides endpoints for creating, viewing, updating, and deleting items. When retrieving a list of items, we also implemented several filtering and sorting options, allowing users to search by category, price range, location, and listing type, and to sort the results by price or by auctions that are ending soon.

5.5. Auctions and Bids

The auction and bidding system is vital to our application. It handles the entire lifecycle of an auction, from the placement of bids to the determination of a winner.

A key component of this system is a background service that we implemented to monitor active auctions. This service runs periodically, checking for auctions whose end time has passed. When an expired auction is found, the service automatically processes it: it identifies the highest bidder, declares them the winner, and records the final results in the database. This automated process ensures that auctions are concluded reliably and without manual intervention. For administrative purposes, we also created an endpoint that allows an admin to end an auction prematurely.

When a user places a bid, the system performs a series of critical validations. First, it confirms that the auction is still active and that the user is not bidding on their own item. Most importantly, it checks if the bid amount is higher than the current highest bid. We implemented a dynamic minimum bid increment, which increases as the item's price gets higher, to ensure meaningful bidding progression.

If an item has a “Buy It Now” price, and a user places a bid that meets or exceeds this price, the auction immediately ends, and that user is declared the winner.

Upon a successful bid, the system updates the item’s current price and number of bids. It also triggers real-time notifications to inform the seller of the new bid and to alert any previous high-bidder that they have been outbid. This ensures all relevant parties are kept up-to-date instantly.

5.6. Search

To help users find items, we have implemented a search and filtering system. This functionality is handled by a single API endpoint that takes care of item retrieval, and it allows for a combination of search parameters to narrow down the results.

The search bar in the frontend uses the free-text search capability that allows users to input a query term. This term is then matched against both the item’s name and its description, making it easy for users to find items.

In addition to the text search, we have included several filters:

- **Category:** Users can filter items to see only those belonging to a specific category.
- **Price Range:** Users can specify a minimum and maximum price to fit their budget.
- **Listing Type:** Users can choose to view only auction items or `fixed_price` items.
- **Location:** Users can filter for items available in a specific city.

Once the search and filtering are applied, the results can be sorted. We have provided options to sort by price (ascending or descending) and, for auctions, by which items are ending soonest.

Our current implementation fetches a list of all items from the database and then applies the filtering and sorting logic in the backend’s memory. This approach is simple and effective for the scale of this project. For a larger-scale application, we would optimize this by building dynamic SQL queries to perform the filtering and sorting directly in the database.

5.7. Real-Time Notifications

To create a realistic experience, we built a real-time notification system. This system is responsible for instantly alerting users to important events, ensuring they are always up-to-date with activities relevant to them.

We identified several key events that should trigger a notification:

- **For sellers:** When their item is sold, or when a new bid is placed on their auction.
- **For bidders:** When they are outbid, when an auction they participated in is won, or when they lose an auction.
- **For messaging:** When a user receives a new message.

Our implementation is centered around a `NotificationService` that works in together with the `WebSocketService`. When an event occurs, such as a new bid, the relevant service logic (e.g., `BidService`) creates a `Notification` object. This object is then saved to the database, providing a persistent record of all notifications for a user.

Immediately after saving the notification, the `NotificationService` calls the `WebSocketService` to broadcast the notification in real-time to the specific user it’s intended for. This is achieved by sending a message over the user’s active `WebSocket` connection. The frontend client, upon receiving this message, can then display an immediate, noticeable alert to the user.

This dual approach ensures reliability. The database storage means users can view a history of their notifications at any time, even if they were offline when the event occurred. The `WebSocket` integration provides the instant feedback that is crucial for a bidding platform.

5.8. Real-Time Messaging

Our messaging system is a real-time one allowing for instant communication between users while also implementing message deletion which is real-time as well.

The system uses ChatRooms, which are created when one user first messages another. For privacy, users can only start conversations with those they have transacted with, either as a buyer or seller. Administrators are an exception and can message any user on the platform.

When a message is sent, it is first validated for content and length. It is then saved to the database and broadcasted in real-time to the recipient using our WebSocketService.

The messaging feature is also integrated with our notification system. When a new message is received, a notification is created and sent. This alerts users to new messages even when they are not in the chat window.

Our messaging system leverages WebSockets for real-time communication, similar to our notification system. When a user sends a message, the MessageService handles the storage of this message in the database, ensuring persistence. Concurrently, the WebSocketService is utilized to broadcast the new message to the intended recipient(s) in real-time. This ensures that messages appear almost instantaneously in the recipient's chat interface.

Key aspects of the messaging implementation include:

- **Persistent Storage:** All messages are stored in the database, allowing users to access their message history at any time.
- **Real-Time Delivery:** WebSockets enable immediate delivery of messages, providing a fluid and interactive chat experience.
- **User-Specific Channels:** Messages are routed to specific users based on their active WebSocket connections, ensuring privacy and direct communication.
- **Message History Retrieval:** Users can retrieve past conversations, which is essential for tracking discussions and agreements.

This real-time messaging capability significantly improves the interactivity of the platform, allowing users to communicate effectively about items, bids, and other transaction-related topics.

5.9. Recommendation System

Our approach for the recommendation system is based on Matrix Factorization. The core idea is to represent the sparse user-item interaction data as a matrix, which we then decompose into two lower-dimensional matrices. One matrix represents latent features for each user (user factors), and the other represents latent features for each item (item factors).

We first collect user interaction data having setup a tracker like service to track user's views on each category, assigning different weights to various activities to signify their importance:

- **Purchases:** Given the highest weight (5.0).
- **Bids:** Given a significant weight (3.0).
- **Category Views:** Given the lowest weight (1.0).

This data is then used to train the model. The training process is an iterative optimization that aims to find the user and item factor matrices that, when multiplied together, best approximate the original user-item interaction matrix. We implemented this using an algorithm based on stochastic gradient descent.

The core of the training logic is in our `matrix_factorization` function. It initializes the user and item factor matrices with small random values and then repeatedly iterates through all known interactions, adjusting the factors to minimize the prediction error.

Here is a snippet of the main training loop:

```
for iteration in 0..ITERATIONS {
  for i in 0..num_users {
    for j in 0..num_items {
      if matrix[i][j] > 0.0 { // If there is an interaction
        let prediction = self.dot_product(&user_factors[i],
&item_factors[j]);
        let error = matrix[i][j] - prediction;

        // Update factors using gradient descent
        for f in 0..LATENT_FACTORS {
          let user_feature = user_factors[i][f];
          let item_feature = item_factors[j][f];

          user_factors[i][f] += LEARNING_RATE
            * (error * item_feature - REGULARIZATION * user_feature);
          item_factors[j][f] += LEARNING_RATE
            * (error * user_feature - REGULARIZATION * item_feature);
        }
      }
    }
  }
}
```

In this loop, `LEARNING_RATE` controls the step size of our adjustments, and `REGULARIZATION` is a term that helps prevent the model from overfitting to the training data.

Once the model is trained, we can predict a user's potential interest in any item by computing the dot product of that user's factor vector and the item's factor vector.

```
fn predict_rating(
  &self,
  model: &MatrixFactorizationModel,
  user_idx: usize,
  item_idx: usize,
) -> f32 {
  self.dot_product(&model.user_factors[user_idx], &model.item_factors[item_idx])
}

fn dot_product(&self, a: &[f32], b: &[f32]) -> f32 {
  a.iter().zip(b.iter()).map(|(x, y)| x * y).sum()
}
```

The trained model is saved to a file to persist across application restarts. We also included an endpoint for administrators to trigger retraining. When a user requests recommendations, the system returns the highest-scoring items they haven't interacted with. If a user has no history, the system falls back to showing the most recently created items.

6. Frontend

The frontend application, built with React and TypeScript, functions as the client-side interface for the eBayish platform. It interacts with the backend's RESTful API. Vite is utilized as the build tool.

6.1. Frontend Architecture

The frontend codebase is structured with key directories organizing different functionalities:

- components: Reusable UI elements.

- pages: Top-level views corresponding to application routes.
- services: API interaction logic.
- contexts: Global application state management.
- hooks: Custom React hooks for reusable stateful logic.
- types: TypeScript interfaces and types for data structures.
- utils: General utility functions.

For UI components and styling, the project leverages Shadcn UI and Tailwind CSS.

6.2. State Management

Global application state, including user authentication status, notifications, and cart contents, is managed using React's Context API. The AuthContext provides authentication-related state and functions:

```
// frontend/src/contexts/AuthContext.tsx
import { createContext, useState, useEffect, useContext, ReactNode } from 'react';
// ... other imports and interfaces ...

const AuthContext = createContext<AuthContextType | undefined>(undefined);

export function AuthProvider({ children }: AuthProviderProps) {
  // ... state and logic ...
  const value = {
    isAuthenticated: !isLoading && !!user,
    user,
    token,
    isLoading,
    login,
    logout,
    setUser,
  };

  return <AuthContext.Provider value={value}>{!isLoading && children}</AuthContext.Provider>;
}

export const useAuth = () => {
  const context = useContext(AuthContext);
  if (context === undefined) {
    throw new Error('useAuth must be used within an AuthProvider');
  }
  return context;
};
```

Local component state is managed using React's useState and useReducer hooks.

6.3. Routing

Client-side navigation is handled by React Router. The BrowserRouter is used in main.tsx. Routes are defined within the App.tsx component or its children to map URL paths to specific pages components.

```
// frontend/src/main.tsx snippet
import { StrictMode } from 'react';
import { createRoot } from 'react-dom/client';
import { BrowserRouter as Router } from 'react-router-dom';
import App from './App.tsx';
```

```

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <Router>
      <App />
    </Router>
  </StrictMode>
);

```

Authorization is enforced through protected routes. Conditional rendering logic restricts access to specific pages, such as the /admin dashboard, for non-admin users. UI elements are dynamically enabled or disabled based on the user's role and authentication status.

6.4. API Integration

The frontend interacts with the backend via a service layer in `frontend/src/services`. This layer abstracts HTTP requests and WebSocket communication.

- `cartService.ts`: Manages user shopping cart interactions.
- `chatService.ts`: Handles messaging API calls.
- `connectionService.ts`: Manages WebSocket connection lifecycle.
- `WebSocketService.ts`: Provides a wrapper for WebSocket communication.

HTTP requests are made using `axios`. A centralized error handling mechanism processes and displays API errors to the user. An example from `cartService.ts` demonstrates an API call:

```

// frontend/src/services/cartService.ts snippet
import axios from 'axios';
// ... interfaces ...

class CartService {
  private baseUrl = '/api';

  async getCart(): Promise<CartItem[]> {
    try {
      const response = await axios.get<CartItem[]>(`${this.baseUrl}/cart`);
      return response.data;
    } catch (error) {
      console.error('Failed to fetch cart:', error);
      throw error;
    }
  }
  // ... other methods ...
}

export const cartService = new CartService();

```

6.5. Real-Time Features

Real-time capabilities are powered by WebSockets.

6.5.1. WebSocket Implementation

The `WebSocketService.ts` module establishes and maintains a WebSocket connection with the backend. It provides methods for sending and receiving messages.

```

// Example: WebSocketService snippet (conceptual)
class WebSocketService {
  private ws: WebSocket | null = null;

```

```
// ...
connect(token: string) {
  this.ws = new WebSocket(`wss://localhost:8000/ws?token=${token}`);
  this.ws.onmessage = (event) => {
    // Process incoming real-time data
  };
}
// ...
}
```

6.5.2. Real-Time Notifications

The frontend's `Notifications.tsx` component, with `WebSocketService`, listens for incoming notification events. Notifications are displayed to the user upon reception.

6.5.3. Real-Time Messaging

The `Messaging.tsx` page and `chatService.ts` handle instant communication. Messages are transmitted via `WebSocket` to the backend, then broadcast to the recipient. The chat interface updates in real-time, displaying new messages and reflecting message deletions.

6.6. Key Feature Implementations

6.6.1. Authentication Flow

The `Login.tsx` and `Signup.tsx` pages manage user authentication. Client-side validation is performed. Upon successful login, the JWT from the backend is stored in `sessionStorage` and attached to authenticated API requests. The `Profile.tsx` page allows users to view and manage account details.

6.6.2. Item Listing and Management

The `CreateListing.tsx` page provides a form for new items, including name, description, price, categories, listing type, and image uploads. Image uploads send files to the backend. `EditListing.tsx` and `ListingManagement.tsx` components enable users to modify and oversee listings.

6.6.3. Marketplace and Search

The `Marketplace.tsx` component presents an interface for browsing items. It integrates a free-text search bar and filtering options: category selection, price range sliders, listing type toggles, and location-based search. These parameters are sent to the backend to refine item retrieval, with results displayed and sortable by criteria such as price or auction end time.

6.6.4. Bidding and Buying

The `ItemDetail.tsx` page facilitates user interaction with items. For auction items, a bidding interface allows users to place bids, respecting dynamic minimum increments and "Buy It Now" options. The `MyBids.tsx` page displays a user's bids. The `Cart.tsx` component manages items for fixed-price purchase.

6.6.5. Admin Dashboard

The `frontend/src/pages/admin` directory houses components for administrative functionalities. This includes interfaces for user management and oversight of platform activities. Allowing the admin to manage users and listing while also having the ability to retrain the recommendation algorithm.